CRM DE ATENDIMENTO E ORÇAMENTOS

Entendendo o Backend

Parte 1 — Fundação e Domínio

O que foi feito, por que foi feito e como conferir com seus próprios olhos

1 2 3 4 5

Parte 1 de 5 do backend

VISÃO GERAL

Onde estamos no plano do backend

1

Fundação + Domínio

Regras do negócio em Python puro, com testes.

VOCÊ ESTÁ AQUI

2

Casos de uso

Orquestram as ações: cadastrar contato, mover card.

3

Persistência

PostgreSQL, SQLAlchemy e Alembic (migrations).

4

API REST

Rotas FastAPI que o frontend vai chamar.

5

Autenticação

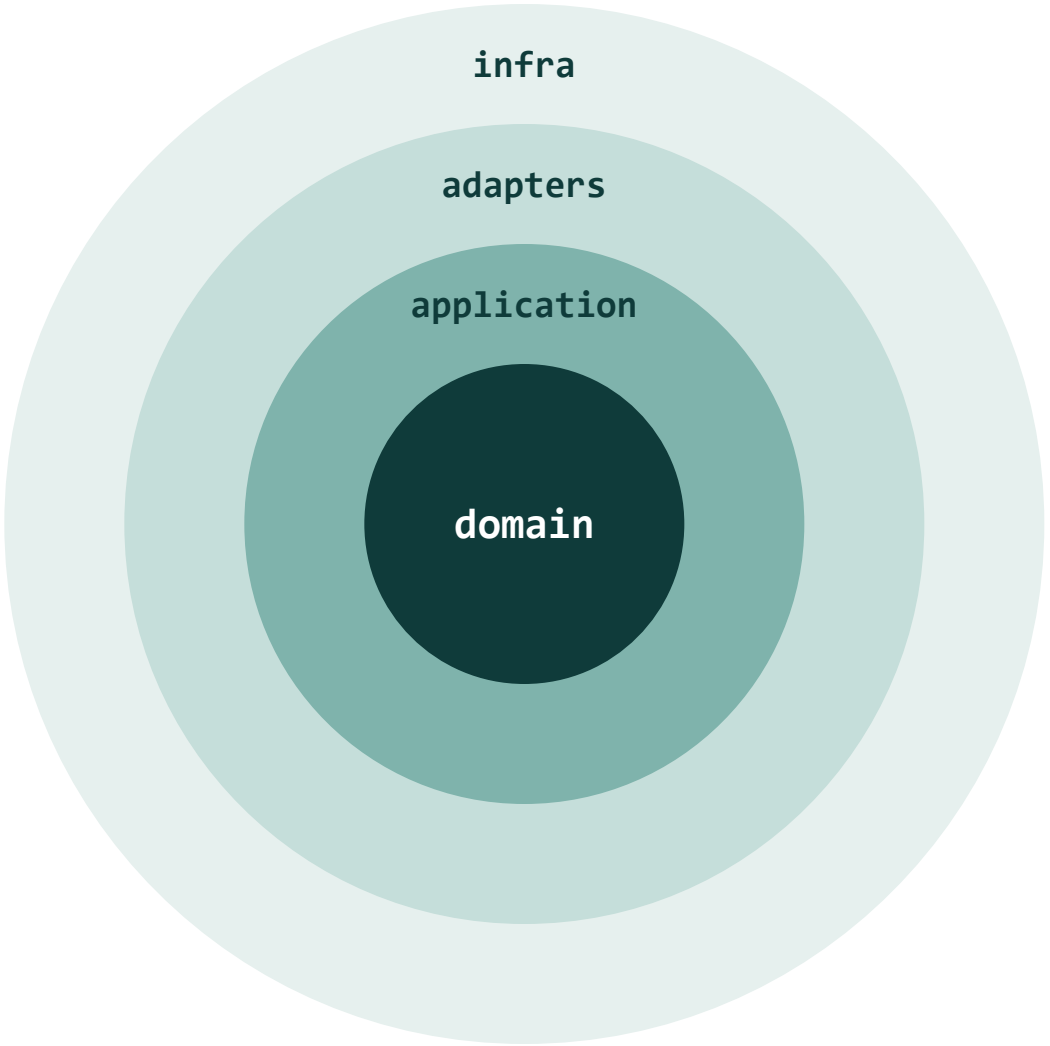
Login com JWT e gestão de usuários.

Ordem definida na documentação (doc 13): domínio → aplicação → infraestrutura → interface. Nunca começar pelo banco nem pela rota.

Por isso a Parte 1 é só Python puro: nenhum banco de dados, nenhum servidor web e nenhuma biblioteca externa dentro do código.

O DESENHO

Clean Architecture: quatro camadas



`domain/` ✓ Parte 1

O coração: as regras do negócio (Orçamento, Telefone, estágios). Não sabe que existe banco ou web.

`application/` Parte 2

Casos de uso: “registrar novo contato”, “mover estágio”. Usa o domínio.

`adapters/` Partes 3–4

Tradutores entre o mundo de fora e o domínio: rotas HTTP e repositórios do Postgres.

`infra/` Partes 3–5

Ferramentas concretas: FastAPI, SQLAlchemy, configuração e JWT.

Regra de ouro: as dependências apontam para dentro. Quem está no centro não importa nada de quem está fora — e um teste automático vigia isso.

MOTIVAÇÃO

Por que começar pelo domínio?

0,2 s

para rodar todos os 114 testes do domínio

Sem banco, sem servidor, sem internet.

1

As regras mais importantes vivem aqui

Não pular etapa e não fechar orçamento sem valor (RF02, RF03).

2

Testa sem banco e sem internet

É Python puro: roda em milissegundos, em qualquer máquina.

3

Trocar tecnologia não quebra a regra

Se amanhã o banco ou o framework mudar, o domínio continua igual (RNF03).

Analogia: a receita do bolo (domínio) não depende de qual forno (banco) ou cozinha (web) você usa. Dá para conferir a receita antes de comprar o forno.

O QUE FOI CRIADO

A estrutura de pastas do backend

```
backend/
├─ pyproject.toml     ficha do projeto + dependências
├─ pytest.ini         configuração dos testes
├─ .venv/             ambiente Python local
├─ src/
│  ├─ domain/         ✓ preenchido nesta parte
│  │  ├─ exceptions.py
│  │  ├─ value_objects/
│  │  ├─ entities/
│  │  └─ repositories/  (Parte 2)
│  ├─ application/    (Parte 2)
│  ├─ adapters/       (Partes 3–4)
│  └─ infra/          (Partes 3–5)
└─ tests/
   └─ unit/
      ├─ test_arquitetura.py
      └─ domain/      testes das regras
```

__init__.py

Arquivo vazio que transforma a pasta em pacote Python. É o que permite escrever from domain.entities.orcamento import Orcamento.

Pastas vazias de propósito

application, adapters e infra já existem para fixar o desenho da arquitetura. Serão preenchidas nas próximas partes.

.venv/

Ambiente Python isolado do projeto, com pytest, mypy e ruff instalados. Está no .gitignore: cada pessoa cria o seu.

FERRAMENTAS

As quatro peças do ferramental

pytest

Roda os testes automáticos. Cada arquivo test_*.py verifica regras do sistema e diz o que passou e o que falhou.

```
python -m pytest
```

mypy --strict

Confere os tipos sem executar o código: pega “passou texto onde era número” antes de rodar (RNF05).

```
python -m mypy
```

ruff

Lint e formatação: padroniza o estilo (imports, tamanho de linha) e aponta código suspeito.

```
python -m ruff check
```

pyproject.toml

A ficha do projeto: nome, versão, dependências (grupo dev) e a configuração do mypy e do ruff.

```
python -m pip install --group dev
```

Os comandos rodam dentro de backend/ — o prefixo completo é .venv\Scripts\python.exe -m ...

COMO FOI FEITO

TDD: o teste vem antes do código



1. Escreve o teste primeiro

Criamos 5 arquivos de teste descrevendo as regras. Ao rodar: ModuleNotFoundError — o código ainda não existia. Falhar aqui é o esperado!

2. Escreve o código para passar

Implementamos exceções, Value Objects e entidades até o pytest mostrar 114 passed.

3. Melhora sem quebrar

O ruff format padronizou o estilo e o mypy conferiu os tipos. Os testes continuaram verdes.

Por que assim? O teste vira a especificação executável da regra. Se no futuro alguém quebrar “não fechar sem valor”, um teste fica vermelho na hora.

CONCEITO

Value Objects: valores que se validam sozinhos

Um valor pequeno que se valida no momento em que nasce. Se for inválido, ele simplesmente não chega a existir.

1. **Imutável**
   @dataclass(frozen=True): depois de criado, não muda mais.

2. **Igual pelo valor**
   Dois telefones com os mesmos dígitos são iguais, mesmo escritos com máscaras diferentes.

3. **Validado na construção**
   O resto do sistema pode confiar: se existe um Email, ele é válido.

Como uma nota de R$ 10: não importa qual nota é, importa o valor. E nota falsa não entra no caixa.

Você cria...	Resultado
`Telefone("(31) 99876-5432")`	✓ "31998765432"
`Telefone("99876-5432")`	✗ FormatoInvalido (sem DDD)
`Email("Joao@Empresa.COM")`	✓ "joao@empresa.com"
`Email("sem-arroba.com")`	✗ FormatoInvalido
`ValorOrcado(Decimal("99.9"))`	✓ 99.90
`ValorOrcado(Decimal("0"))`	✗ ValorInvalido (≤ 0)

Também são Value Objects: EstagioAtendimento e TipoInteracao (enums — conjunto fechado de opções).

POR DENTRO DO CÓDIGO

Exemplo: como o Telefone se valida

```python
@dataclass(frozen=True)                    # (1)
class Telefone:
    numero: str

    def __post_init__(self) -> None:       # (2)
        digitos = tirar_mascara(self.numero)
        if not TELEFONE_BR.fullmatch(digitos):
            raise FormatoInvalido(         # (3)
                "Telefone inválido...",
                campo="telefone",
            )
        # (4) guarda só os dígitos
        object.__setattr__(self, "numero", digitos)
```

1 **`frozen=True`**

Torna o objeto imutável. Tentar alterar numero depois dá erro.

2 **`__post_init__`**

Método que o Python chama logo após criar o objeto: é aqui que validamos.

3 **`raise FormatoInvalido`**

Formato errado? Lança o erro e o objeto não é criado. O campo diz qual dado corrigir.

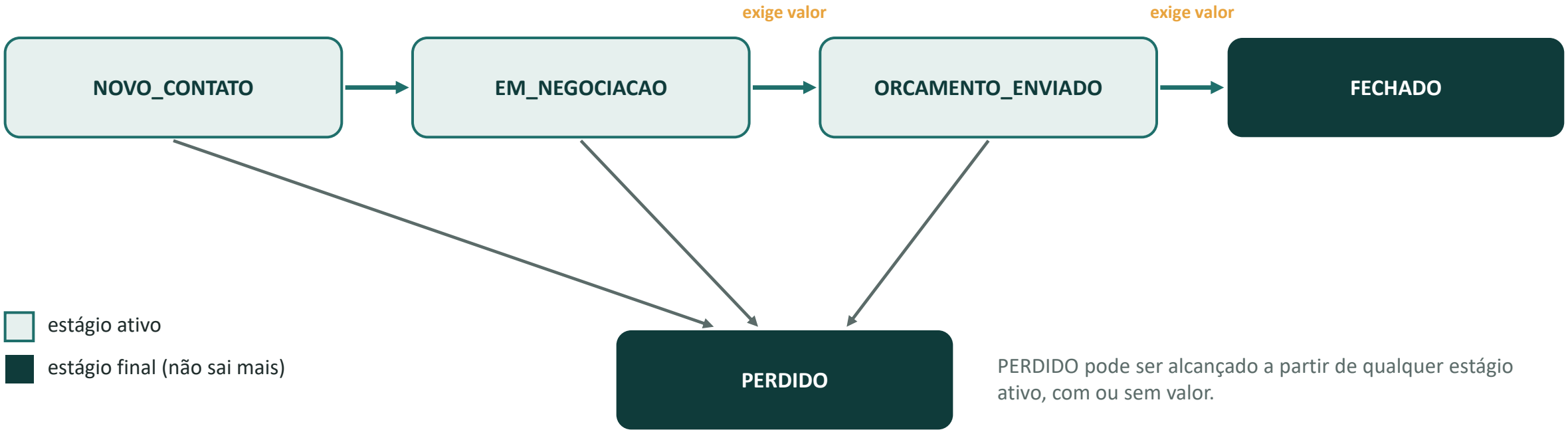
4 **`Guarda só os dígitos`**

“(31) 99876-5432” vira “31998765432”. A máscara é assunto da tela (RN04).

Regra: DDD com 2 dígitos (1 a 9) + 8 dígitos (fixo) ou 9 dígitos começando com 9 (celular). Versão simplificada — o código real está em src/domain/value_objects/telefone.py

A REGRA CENTRAL

EstagioAtendimento: o funil como máquina de estados



PERDIDO pode ser alcançado a partir de qualquer estágio ativo, com ou sem valor.

No código: mapa TRANSICOES_PERMITIDAS em estagio_atendimento.py. Os testes cobrem as 25 combinações (5×5): 6 permitidas e 19 proibidas.

✗ Pular etapa

`NOVO_CONTATO → FECHADO`

Rejeitado com TransicaoInvalida

✗ Voltar

`ORCAMENTO_ENVIADO → EM_NEGOCIACAO`

Rejeitado com TransicaoInvalida

✗ Reabrir

`FECHADO / PERDIDO → qualquer um`

Rejeitado com TransicaoInvalida

Orcamento: quem aplica as regras

```
def mover_estagio(self, novo, hoje):
    # 1) essa transição existe no mapa?
    if not self.estagio.pode_mover_para(novo):
        raise TransicaoInvalida(..., campo="estagio")

    # 2) o destino exige valor e não tem?
    if novo.exige_valor() and self.valor is None:
        raise TransicaoInvalida(..., campo="valor")

    # 3) passou nas duas: agora sim, muda
    self.estagio = novo
    if novo.eh_terminal():          # FECHADO ou PERDIDO
        self.data_fechamento = hoje
```

`abrir(cliente_id, hoje)`

Nasce em NOVO_CONTATO, sem valor.

`mover_estagio(novo, hoje)`

Aplica as regras de transição.

`definir_valor(valor)`

Define o preço; bloqueado se encerrado.

`registrar_interacao(...)`

Adiciona um contato ao histórico.

`esta_em_aberto()`

True se não for FECHADO nem PERDIDO.

Ponto-chave: todas as verificações acontecem ANTES de mudar qualquer coisa. Se der erro, o orçamento fica exatamente como estava — nunca pela metade (RNF04). Por isso ele é a “raiz do agregado”: toda mudança passa por ele.

AS OUTRAS ENTIDADES

Cliente, Interação e Usuário

C

Cliente

`cliente.py`

- Raiz de agregado: o “dono” do contato.
- Só existe com nome, Telefone e Email válidos (RF09).
- abrir_orcamento() cria um Orçamento em NOVO_CONTATO.
- Pode ter vários orçamentos ao longo do tempo (RF08).
- Empresa e tipo de embalagem são opcionais.

I

Interação

`interacao.py`

- Registro de um contato: MENSAGEM, LIGACAO ou REUNIAO.
- Imutável: é um fato que já aconteceu.
- Só nasce via orcamento.registrar_interacao() — vive dentro do Orçamento (composição).
- Conteúdo vazio é rejeitado.

U

Usuário e Administrador

`usuario.py`

- Administrador é um Usuário (herança) com perfil ADMINISTRADOR.
- Pode ativar e desativar outros usuários.
- Não pode desativar a própria conta (RF10).
- Login e senha ficam para a Parte 5.

Por que Cliente e Orçamento são separados? Um cliente pode acumular anos de orçamentos. Ligando os dois só pelo cliente_id, mover um card não precisa carregar todo o histórico do cliente.

QUANDO ALGO DÁ ERRADO

Exceções de domínio: erros com nome e endereço

Exceção	Quando acontece	campo
ErroDominio	Base (mãe) de todas — carrega mensagem e campo	—
TransicaoInvalida	Pular/voltar etapa, sair de estágio final, mover sem valor	`estagio` · `valor`
FormatoInvalido	Telefone, e-mail, nome ou conteúdo inválido	`telefone` · `email` · `nome` · `conteudo`
ValorInvalido	Valor zero, negativo ou com mais de 2 casas decimais	`valor`
OperacaoNaoPermitida	Administrador tentando desativar a própria conta	`ativo`

Por que o campo? Na Parte 4 a API vai converter a exceção neste JSON (status 422) e o frontend usa o campo para destacar o input certo.

```
{
  "erro": "TransicaoInvalida",
  "mensagem": "Não é possível mover de NOVO_CONTATO para FECHADO",
  "campo": "estagio"
}
```

Teste de arquitetura: o computador vigia a regra

1 **Lê cada arquivo .py**

de src/domain/ e src/application/.

2 **Lista os imports com ast**

ast é o analisador de código do próprio Python: lê o texto, não executa nada.

3 **Compara com a lista proibida**

fastapi, sqlalchemy, pydantic, alembic, jwt... e camadas externas (adapters, infra).

O experimento que fizemos

```
# 1) colocamos no topo de orcamento.py:
import fastapi

> pytest tests/unit/test_arquitetura.py
FAILED  Imports proibidos em domain/:
  {'domain\entities\orcamento.py': ['fastapi']}

# 2) removemos a linha e rodamos de novo:
> pytest
114 passed
```

Resultado: a regra “o domínio não depende de framework” (RNF03) deixa de ser só uma promessa no documento — o computador vigia. Se alguém quebrar, o teste fica vermelho.
O próprio verificador também tem testes: 9 casos garantem que ele detecta o que deve e não acusa o que é permitido.

RESULTADO

Como a Parte 1 terminou

114

testes passando

em cerca de 0,2 segundo

25

combinações de estágio testadas

6 permitidas · 19 proibidas

0

erros no mypy --strict

tipos conferidos em src e tests

0

avisos do ruff

código no padrão de estilo

Critério de pronto do plano atingido: pytest verde, mypy limpo e o teste de arquitetura reprova um import fastapi no domínio.

MÃO NA MASSA

Como rodar e explorar você mesmo

```
cd backend

# só na primeira vez (instala pytest, mypy, ruff)
.venv\Scripts\python.exe -m pip install --group dev

.venv\Scripts\python.exe -m pytest        # roda tudo
.venv\Scripts\python.exe -m pytest -v     # nome de cada teste
.venv\Scripts\python.exe -m mypy          # checa os tipos
.venv\Scripts\python.exe -m ruff check    # estilo
```

Ordem sugerida de leitura

1. **`estagio_atendimento.py`**
   o mapa do funil
2. **`orcamento.py`**
   as regras de transição
3. **`test_orcamento.py`**
   as regras viram testes
4. **`telefone.py · email.py`**
   as validações

Exercício para fixar

1. Em estagio_atendimento.py, acrescente EstagioAtendimento.FECHADO ao conjunto de NOVO_CONTATO.
2. Rode pytest -v e veja quais testes ficam vermelhos — e leia a mensagem de cada um.
3. Desfaça a mudança (Ctrl+Z) e rode de novo: tudo verde.

PARA REVISAR

Glossário rápido e próximo passo

Domínio	As regras do negócio, escritas em código puro.
Entidade	Objeto com identidade (id) que muda com o tempo: Orçamento, Cliente.
Value Object	Valor sem id, imutável, validado ao nascer: Telefone, Email.
Aggregate Root	A porta de entrada de um grupo: Interação só muda via Orçamento.
TDD	Escrever o teste antes do código: red → green → refactor.
Clean Arch.	Camadas com dependências apontando para dentro, domínio no centro.

PRÓXIMO PASSO

Parte 2 — Casos de uso

- Interfaces de repositório: as “tomadas” onde o banco vai ser ligado.
- Repositórios falsos em memória para testar sem banco.
- RegistrarNovoContato, MoverEstagio, DefinirValor, RegistrarInteracao, Métricas e Histórico.
- Teste-chave: se o domínio recusar, nada é salvo (RNF04).